

Sprint 1 (Semanas 1-2): O Alicerce e o Núcleo de Decisão

Meta: Ter o *backend* (Nest.JS) e o *frontend* (React) configurados. O Admin (Carlos) deve conseguir **criar uma política (Tela 1)** e **salvá-la no banco de dados**.

- **Epic (Backend):** Configurar o Alicerce (DevOps & DB).
 - **Tarefa 1.1 (DevOps):** Iniciar os repositórios (Backend/Frontend)
 - **Tarefa 1.2 (DevOps):** Configurar o docker-compose.yml (para Nest.JS, PostgreSQL, Redis).
 - **Tarefa 1.3 (DB):** Implementar o schema.prisma (Artefacto #1) e rodar a primeira migração.
 - **Epic (Backend):** Implementar o Ω (L2).
 - **Tarefa 1.4 (Eng):** Criar o Módulo Nest.JS para o Ω (Seção 3.4).
 - **Tarefa 1.5 (Eng):** Integrar a biblioteca **CEL** ou **JSONLogic** (Seção 7.3).
 - **Tarefa 1.6 (QA):** Escrever Testes de Unidade (Seção 6.2) para o Ω (ex: "dado 'metrics' X, retorna 'BLOCK'").
 - **Epic (API):** Implementar a API de Políticas (Artefacto #2).
 - **Tarefa 1.7 (Eng):** Implementar GET /api/v1/policies/{domain}.
 - **Tarefa 1.8 (Eng):** Implementar POST /api/v1/policies/{domain}.
 - **Epic (Frontend):** Construir a Tela 1.
 - **Tarefa 1.9 (Eng):** Construir o "Editor de Políticas" (React, Tela 1) que consome as APIs (1.7, 1.8).
-

Sprint 2 (Semanas 3-4): O Pilar de Auditoria (O "Ouro")

Meta: Ter o *pipeline* de log funcionando. Um *prompt* bloqueado (BLOCK) deve **aparecer no Painel de Auditoria (Tela 2)** com o *status* tsa_status: "pending".

- **Epic (Backend):** Implementar Kai (L0) e MIR (L1).
 - **Tarefa 2.1 (Eng):** Implementar o Módulo Kai (Seção 3.2), incluindo a mitigação **RE2** (Seção 7.5).
 - **Tarefa 2.2 (Eng):** Implementar o Módulo MIR (Seção 3.3), com a arquitetura de *plugin* (usar um *mock* de API de sentimento) e a telemetria de latência (Seção 7.6).
- **Epic (Backend):** Implementar o Ledger (L4).
 - **Tarefa 2.3 (Eng):** Criar o Módulo Ledger (Seção 3.6).
 - **Tarefa 2.4 (Eng):** Implementar a fila de *retry* (Seção 7.4) usando **Redis**.

- **Tarefa 2.5 (Eng):** Implementar a integração real com a **TSA** (Seção 7.2) — *Esta é a tarefa mais crítica do sprint.*
 - **Epic (API):** Implementar o *Pipeline* e a Auditoria (Artefacto #2).
 - **Tarefa 2.6 (Eng):** Implementar POST `/api/v1/chat/invoke` (o *pipeline* completo v9.0).
 - **Tarefa 2.7 (Eng):** Implementar GET `/api/v1/logs` e GET `/api/v1/logs/{id}`.
 - **Epic (Frontend):** Construir a Tela 2.
 - **Tarefa 2.8 (Eng):** Construir o "Painel de Auditoria" (React, Tela 2) que consome as APIs (2.7).
-

Sprint 3 (Semanas 5-6): O Loop de Correção e Validação do MVP

Meta: Ter o *pipeline* REVIEW e o *switch* de debug funcionando. O Auditor (Ana) deve conseguir **ver o texto original bloqueado (debug)** na Tela 2. O Operador (Bruno) deve conseguir **resolver um item na Tela 3**.

- **Epic (Backend):** Implementar Retro (L3) e Reviser.
 - **Tarefa 3.1 (Eng):** Criar o Módulo Retro (Seção 3.5).
 - **Tarefa 3.2 (Eng):** Implementar a lógica `buildPrompt()` do Reviser (Seção 4.5).
 - **Tarefa 3.3 (Eng):** Implementar o *loop* de re-verificação do Reviser (Seção 4.6).
 - **Epic (Backend):** Implementar o *Switch* de Validação (O seu pedido).
 - **Tarefa 3.4 (Eng):** Implementar o `logging.level: "debug"` no *pipeline* `chat/invoke` para salvar o `raw_text_input` no AuditLog (Schema, Artefacto #1).
 - **Epic (API):** Implementar a API de Revisão (Artefacto #2).
 - **Tarefa 3.5 (Eng):** Implementar os *endpoints* GET `/api/v1/review-queue` e POST `/api/v1/review-queue/{logId}/...`
 - **Epic (Frontend):** Construir a Tela 3 e atualizar a Tela 2.
 - **Tarefa 3.6 (Eng):** Construir a "Fila de Revisão" (React, Tela 3).
 - **Tarefa 3.7 (Eng):** Atualizar o "Painel de Detalhes" (Tela 2) para mostrar o `raw_text_input` se ele existir.
-

Sprint 4 (Semanas 7-8): V&V, Endurecimento e Deploy

Meta: Atingir TRL 7. O *pipeline* está completo e validado pelo *Golden Set*. O MVP está pronto para a demonstração

- **Epic (QA):** Executar a Validação (V&V).
 - **Tarefa 4.1 (QA):** Criar o *script* de V&V (Seção 6.3) que roda o *Golden Set* contra a API chat/invoke.
 - **Tarefa 4.2 (Eng):** Corrigir todos os *bugs* do V&V (ex: falhas no FPR, Seção 6.4).
 - **Tarefa 4.3 (Compliance):** Implementar o versionamento do *Golden Set* (Seção 6.1).
- **Epic (DevOps):** Configurar o CI/CD.
 - **Tarefa 4.4 (DevOps):** Configurar o **GitHub Actions** para rodar os Testes de Unidade e o *script* de V&V a cada *commit*.
- **Epic (Produto):** Polimento Final.
 - **Tarefa 4.5 (Eng):** Implementar o RBAC (Role-Based Access Control) nos *endpoints* da API (Seção 5.4).
 - **Tarefa 4.6 (Eng):** Implementar a exportação para PDF/A (Seção 6.1) na Tela 2.
 - **Tarefa 4.7 (DevOps):** Preparar o *deploy* final

ARTEFATO 01

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum Decision {
  ALLOW
  REVIEW
  BLOCK
}

enum TsaStatus {
  pending
  confirmed
  failed
}

enum Role {
  ADMIN_COMPLIANCE
  AUDITOR
  OPERATOR_HITL
}

enum Mode {
  FAST
  DEEP
}

model User {
  id      String    @id @default(uuid())
  email   String     @unique
  name    String?
  role    Role       @default(AUDITOR)

  // Multi-tenant (opcional)
  tenantId String?   @db.VarChar(64)

  Policies PolicyConfig[]
  AuditLogs AuditLog[]
}

model PolicyConfig {
  id      String    @id @default(uuid())
```

```

domain String
version Float @default(1.0)
rules Json
createdAt DateTime @default(now()) @db.Timestamptz(6)
updatedAt DateTime @updatedAt @db.Timestamptz(6)

// Quem publicou/alterou
ownerId String?
Owner User? @relation(fields: [ownerId], references: [id])

// Hash canônico desta versão (para PoP-L vincular)
policyHash String @db.VarChar(128)

@@unique([domain, version])
@@index([domain])
@@index([policyHash])
}

model AuditLog {
  id String @id @default(uuid())
  createdAt DateTime @default(now()) @db.Timestamptz(6)

  // Contexto/observabilidade
  correlationId String? @db.VarChar(64)
  conversationId String? @db.VarChar(64)
  tenantId String? @db.VarChar(64)

  // Política efetivamente aplicada
  policyDomain String
  policyVersion Float
  policyHash String @db.VarChar(128) // hash da PolicyConfig no momento

  // Decisão do Ω
  decision Decision
  ruleTriggered String? // ex.: "BLOCK: PII Detectado"
  rulePriority Int?
  ruleVersion Float? // se você versionar regras individualmente
  rationale String?

  // Métricas (entrada consolidada do MIR)
  metrics_input Json

  // Execução / caminho
  mode Mode // FAST ou DEEP
  revised Boolean @default(false)
  revision_count Int @default(0)

  // LLM info / SRE
  llm_provider String? @db.VarChar(64)

```

```

llm_model    String?  @db.VarChar(64)
llm_version  String?  @db.VarChar(32)
latency_ms   Int?     // latência adicional do pipeline

// Integridade criptográfica (PoP-L)
text_hash_input    String  @db.VarChar(128)
text_hash_output_final String? @db.VarChar(128)
payload_hash_sha3_512 String  @db.VarChar(128) // hash do DED (JSON) assinado

// Timestamping (TSA)
tsa_status    TsaStatus @default(pending)
tsa_token     String?   // armazene base64/hex; ou use bytea via @@map/atributos nativos
tsa_authority_uri String?

// Relações
userId    String?
User      User?   @relation(fields: [userId], references: [id])

@@index([policyDomain, createdAt])
@@index([decision, createdAt])
@@index([mode, createdAt])
@@index([policyHash])
@@index([tenantId, createdAt])
@@index([correlationId])
@@index([conversationId])
}

/// Tabela separada para texto bruto do MVP (debug only).
/// Em produção, mantenha vazia/desabilitada por política de retenção.
/// Ideal com RLS + pgcrypto (criptografia por coluna) e TTL curto.
model AuditLogDebug {
  id          String  @id @default(uuid())
  auditLogId   String  @unique
  raw_text_input String? // criptografar com pgcrypto (fora do Prisma)
  raw_text_output String?

  createdAt    DateTime @default(now()) @db.Timestamptz(6)

  AuditLog      AuditLog @relation(fields: [auditLogId], references: [id], onDelete: Cascade)
}

```

Observações:

- **Particionamento:** implemente no Postgres (DDL) por mês/tenant para AuditLog (Prisma não gera isso; use migrations SQL manuais).
- **RLS:** crie políticas por tenantId/role.
- **Hash:** padronize **hex** (64/128 chars) para SHA-256/SHA3-512.
- **TSA:** armazene tsa_token como base64 (string) ou bytea; documente o verificador externo.
- **GIN:** crie índices GIN manuais em metrics_input e rules (migrations SQL).

Por que isso importa (impacto direto)

- **Auditabilidade forte:** policyHash + payload_hash_sha3_512 tornam cada decisão **reproduzível e verificável** por terceiros.
- **Forense pronta:** mode, revised, revision_count, rulePriority, latency_ms → explicam o “como” e “quanto custou”.
- **Segurança:** debug em tabela separada com RLS/TTL; evita vazamento de PII no ledger.
- **Escala:** índices e particionamento mantêm consultas rápidas mesmo com milhões de logs/mês.

Especificação de API v1.1

Princípios Fundamentais (r1.1)

1. **Segurança (RBAC):** Todos os *endpoints* (exceto chat/invoke) são protegidos e requerem autenticação. O *endpoint* de logs terá lógica de acesso especial (ver 3.2).
 2. **Multi-Tenancy:** (Se aplicável) Todos os *queries* são implicitamente filtrados pelo *tenantId* do utilizador autenticado.
 3. **Observabilidade:** Todas as requisições devem passar (ou gerar) um *correlationId* para rastreabilidade (SRE).
-

1. Endpoint de "Chat" (O Pipeline Principal)

Este é o *endpoint* que a aplicação de *chat* do cliente final usará.

- POST /api/v1/chat/invoke
 - **Propósito:** Envia um *prompt* de utilizador e recebe uma resposta segura e governada.
 - **Corpo (Request):** { "prompt": "...", "userId": "user-123", "domain": "finance", "correlationId": "uuid-..." (opcional) }
 - **Processo (Backend r1.1):**
 1. Executa o *pipeline* v9.0 completo (Kai $\rightarrow$ MIR $\rightarrow$ Ω Retro/Reviser).
 - 2. **No Módulo Ledger (L4):**
 - Cria o registo na tabela AuditLog, preenchendo *todos* os novos campos (Seção 7): policyHash, mode, latency_ms, llm_provider, correlationId, etc.
 - Calcula o payload_hash_sha3_512 do DED e o salva.
 - Envia para a fila da TSA (marcando tsa_status: "pending").
 - **(Lógica de Debug):** Se logging.level == "debug", cria um registo *separado* na tabela AuditLogDebug com o raw_text_input.
 - **Corpo (Response):** { "final_text": "...", "audit_log_id": "uuid-...", "correlationId": "uuid-..." }
-

2. Endpoints de "Políticas" (Usados pela Tela 1)

Gerencia o PolicyConfig (Artefacto #1, r1.1).

- GET /api/v1/policies/{domain}
 - **Propósito:** Busca a *versão ativa* (mais recente) de uma política para o Editor (Tela 1).
 - **Corpo (Response):** { "domain": "finance", "version": 1.3, "rules": [...], "policyHash": "sha3-..." }
- POST /api/v1/policies/{domain}
 - **Propósito:** Publica uma **nova versão** da política.
 - **Corpo (Request):** { "rules": [...] }
 - **Processo (Backend r1.1):**
 1. Valida as regras (sintaxe CEL/JSONLogic).
 2. Calcula o policyHash (Seção 7.2) do novo conjunto de regras.
 3. Incrementa a version (ex: 1.3 $\rightarrow$ 1.4).
 4. Salva esta *nova* versão na tabela PolicyConfig. (A versão antiga é mantida para histórico de auditoria).

3. Endpoints de "Auditoria" (Usados pela Tela 2)

O *endpoint* mais crítico, alinhado com o *schema* "blindado".

- GET /api/v1/logs
 - **Propósito:** Busca a lista de *logs* para a tabela principal da Tela 2.
 - **Query Params (Filtros r1.1):** ?date_start=..., ?action=BLOCK, ?policyDomain=..., ?mode=DEEP, ?correlationId=...
 - **Corpo (Response):** [{ "id": "...", "createdAt": "...", "decision": "BLOCK", "rationale": "...", "mode": "DEEP", "tsa_status": "confirmed" }, ...]
- GET /api/v1/logs/{id}
 - **Propósito:** Busca o **Dossiê de Decisão (DED)** completo para o "Painel de Detalhes" (Tela 2).
 - **Processo (Backend r1.1 - A Lógica de Segurança):**
 1. O *backend* busca o *log* principal na tabela AuditLog (o "Dossiê").
 2. O *backend* verifica o role (ex: ADMIN_COMPLIANCE) do utilizador autenticado (via JWT).
 3. **Lógica Condicional:** Se (logging.level == "debug") E (user.role == "ADMIN_COMPLIANCE"):

- O *backend* executa um JOIN (ou uma segunda *query*) na tabela AuditLogDebug usando o auditLogId.
 - O *backend* **anexa** o raw_text_input e raw_text_output ao Dossiê, apenas para esta resposta da API.
4. Se a condição falhar, o Dossiê é retornado *sem* os textos brutos (provando que a privacidade funciona).
- **Corpo (Response):** O JSON completo da Seção 3.6 (r3.1), mas com o raw_text_input **opcionalmente** incluído com base na permissão (RBAC).
-

4. Endpoints de "Revisão" (Usados pela Tela 3)

- GET /api/v1/review-queue
 - **Propósito:** Busca a lista de tarefas para a Tela 3.
- POST /api/v1/review-queue/{logId}/approve
 - **Processo (Backend r1.1):** O *backend* atualiza o AuditLog (ID: {logId}) para revised: true, revision_count: 1.
- POST /api/v1/review-queue/{logId}/edit
 - **Processo (Backend r1.1):** Atualiza o AuditLog e salva o *hash* do novo texto (text_hash_output_final).
- POST /api/v1/review-queue/{logId}/reject
 - **Processo (Backend r1.1):** Atualiza o AuditLog (ID: {logId}) para decision: "BLOCK".

Guia v1.2 — Hardening do TER–Kai (MVP prático)

1) Schema & Enums (fundação de dados)

Objetivo: estabilizar contratos e evitar migrações retrabalhadas.

Mudanças concretas (Prisma):

- Enums:

```
enum Decision { ALLOW REVIEW BLOCK }
enum Mode { FAST DEEP }
enum TsaStatus { queued processing confirmed failed }
```

- PolicyConfig: adicionar policyHash String @db.VarChar(128)
- AuditLog: campos canônicos e indexação:

```
model AuditLog {
  id          String    @id @default(uuid())
  createdAt   DateTime  @default(now())
  policyDomain String    @index
  policyVersion Float
  policyHash   String
  decision     Decision
  ruleTriggered String?
  rationale    String?
  metrics_input Json
  mode         Mode
  latency_ms   Int
  llm_provider String?
  text_hash_input      String
  text_hash_output_final String?
  payload_hash_sha3_512 String @unique
  tsa_status   TsaStatus @default(queued)
  tsa_token    String?
  tsa_authority_uri String?
  correlationId String? @index
  userId       String?
  User         User? @relation(fields: [userId], references: [id])
}
```

- Outbox** (idempotência TSA):

```
model TsaOutbox {
  auditLogId String @id
  status      TsaStatus @default(queued)
  attemptCount Int @default(0)
  lastError   String?
  AuditLog    AuditLog @relation(fields: [auditLogId], references: [id])
}
```

- Debug seguro:**

```

model AuditLogDebug {
  auditLogId String @id
  raw_text_input Bytes // usar pgcrypto/ext ou app-level envelope
  raw_text_output Bytes
  AuditLog AuditLog @relation(fields: [auditLogId], references: [id])
}

```

DoD: migração roda sem erro; seed mínimo (uma policy) salva; tipos Prisma gerados.

Risco: divergência FAST/DEEP (“FAST_PATH” antigo). Corrigido aqui com Mode.

2) Contratos tipados + Schemas (Zod)

Objetivo: travar shape dos objetos do repo todo.

Mudanças:

- Monorepo @shared-types com:
 - MetricsInputFlat (todas as chaves sempre presentes):


```

export const MetricsInputZ = z.object({
  pii_count: z.number(),
  compliance_keywords_found: z.array(z.string()),
  basic_sentiment_score: z.number(),
  extracted_claims: z.array(z.string()),
  plugin_sentiment_score: z.number().nullable().default(null),
  plugin_sentiment_latency_ms: z.number().nullable().default(null),
  plugin_bias_score: z.number().nullable().default(null),
}).strict();
          
```
 - DecisionDossierZ (DED) com normalizações (strings ISO, números toFixed(6) em função utilitária).
- Backend: validar **entrada/saída** de todas as rotas com Zod.
- Frontend: usar os mesmos tipos de @shared-types.

DoD: build passa sem any; 100% das rotas principais validam com Zod.

3) Ω (Engine) estabilizado + testes

Objetivo: cérebro de decisão previsível.

Mudanças:

- Provider de regras (CEL ou JSONLogic) encapsulado em OmegaService.
- Linter de regras: alerta para campos inexistentes e prioridades duplicadas.
- **Tests** (jest): casos PII, keywords, ERROR_TIMEOUT, prioridades.

DoD: `omega.service.spec.ts` verde; cobertura >90% no service.

4) Kai L0 (hardening)

Objetivo: entrada confiável; PII não vaza.

Mudanças:

- RE2 já ok; **hashPII** antes de retornar (sha256 + salt de ambiente):

```
const maskPII = (s:string)=> sha256(s + process.env.PII_SALT).slice(0,32);
detections.push({ type:'email', value: maskPII(email) });
```
- `extract(text, keywordsFromPolicy)`: keywords vêm da policy.
- Normalização segura (lowercase, trim, NFC); truncamento por tamanho.

DoD: “PII bench” com Golden Set inicial; FNR/FPR reportados.

Risco: guardar PII cru em debug. Em MVP, **Bytes criptografados** (envelope) ou mascarar mesmo no debug.

5) MIR L1 (plugins + telemetria + circuit breaker)

Objetivo: orquestrar com custo/latência controlados.

Mudanças:

- Interface de plugin:

```
export interface MetricPlugin {
  name: 'advanced_sentiment'|'bias_check';
  budgetMs: number;
  run(text:string): Promise<number>;
}
```
- Wrapper com timeout/`ERROR_TIMEOUT` e latência.
- `canonicalizeMetricsInput()` → sempre retorna **todas** as chaves, com `null` quando plugin não rodar.

DoD: chamadas com timeout; métricas populadas e validadas (Zod).

6) Retro/Reviser (MVP) + verificação

Objetivo: REVIEW fail-closed.

Mudanças:

- `Retro.correct(text, rationale)` → monta **meta-prompt** específico (REMOVER_PII / AJUSTAR_TOM).
- **LLM-V mínimo** (verificador) em Fast Path:
 - prompts binários JSON: `{violates:boolean, reasons:string[]}`
 - se `violates=true` → BLOCK.
- Re-verificação **sempre** antes de liberar.

DoD: testes de fluxo REVIEW → corrigir → reverificar → ALLOW/BLOCK.

7) Ledger L4 (transação + hash determinístico)

Objetivo: ouro da auditoria sem inconsistências.

Mudanças:

- `prisma.$transaction([create AuditLog, create TsaOutbox, create Debug?])`.
- `stableStringify` do **DED canônico** (após `canonicalizeMetricsInput` + `canonicalizeNumbers` + datas ISO).
- `payload_hash_sha3_512` único.
- `AuditLogDebug`: armazenar **Bytes cifrados** (envelope) ou mascarados.

DoD: o mesmo input gera **sempre** o mesmo `payload_hash_sha3_512` (teste de determinismo no CI, 1000 execs).

8) TSA Worker (idempotência & revalidação)

Objetivo: confirmar PoP-L com segurança.

Mudanças:

- Ler `TsaOutbox`, marcar `processing`, chamar TSA, **recomparar** `payload_hash_sha3_512` do banco antes de `confirmed`.
- Retry/backoff; attempts contados; logs.

DoD: `confirmed` **só** se hash bater; retries automáticos; estados consistentes (`queued` → `processing` → `confirmed/failed`).

9) Roteador de Risco (RBR) & Orquestrador E2E

Objetivo: custo vs risco.

Mudanças:

- `getMode(domainRisk:number, kaiSignals:MetricsInput): Mode`
 - `domainRisk>0.7 || extracted_claims.length>0 || keywords.length>0 → DEEP`
- Tracing com `correlationId`.
- Métricas de latência por estágio.

DoD: cenários high-risk forçam Deep; latências visíveis no painel.

10) APIs finais & RBAC

Objetivo: superfície segura.

Mudanças:

- `/policies`: GET com ETag; POST valida regras com Zod + `policyHash`.
- `/chat/invoke`: aceita `domain`; retorna `final_text` + `logId`.
- `/logs`, `/logs/:id`: **sem PII**; endpoint de verificação TSA server-side.
- RBAC: `ADMIN_COMPLIANCE`, `AUDITOR`, `OPERATOR_HITL`. Debug só com role + `LOGGING_LEVEL=debug`.

DoD: testes de autorização; respostas sem vazamento.

11) Frontend — Tela 1, 2 e 3

Objetivo: operação e visibilidade.

Mudanças:

- Tela 1: editor com validação Zod; mostra `policyHash` e diff.
- Tela 2: lista + detalhe; “TSA: verified/failed”; diff seguro original ↔ revisado (quando debug+RBAC).
- Tela 3: Fila HITL (approve/edit/reject) com justificativa.

DoD: fluxos felizes e erros tratados.

12) Golden Set & CI (V&V contínuo)

Objetivo: prova contínua.

Mudanças:

- `golden_set_v1.0.json` por domínio: PII (obfuscações), legal, tom, adversarial.
- Job CI: roda o set em `/chat/invoke`, verifica decisão, DED gerado, TSA confirmed no Deep.

DoD: pipeline falha se KPIs caírem (ex.: PII detect < 99.9%); relatórios por classe (FNR/FPR).

13) Observabilidade & Runbooks

Objetivo: operar sem adivinhação.

Mudanças:

- Métricas: P50/95/99 por estágio; distribuição ALLOW/REVIEW/BLOCK por regra; TSA failed; lag de fila.
- Traces com `correlationId`.
- Runbooks: queda do plugin, TSA indisponível, regressão de regra, aumento de REVIEW.

DoD: dashboards (Grafana/Datadog) + alertas básicos.

14) Retenção & DLP

Objetivo: fechar privacidade.

Mudanças:

- TTL `AuditLogDebug` (7–30d) + purge job.
- DLP de egress (bloquear exportação de PII).
- Rotação de chaves (envelope): `dek_key_id`, `kek_version`; playbook de rotação.

DoD: simulado em staging; checklist de privacidade assinado.

Patches rápidos (excertos práticos)

A. Canonicalização antes do hash (Node)

```
export function canonicalizeNumbers(obj:any) {
  const f = (v:any):any =>
    typeof v === 'number' && Number.isFinite(v) ? Number(v.toFixed(6))
```



```

      : Array.isArray(v) ? v.map(f)
      : v && typeof v==='object' ?
Object.fromEntries(Object.keys(v).sort().map(k=>[k, f(v[k])]))
      : v;
    return f(obj);
  }

```

B. DED canônico

```

const ded = DecisionDossierZ.parse({
  // ...campos obrigatórios
  metrics_input:
MetricsInputZ.parse(canonicalizeNumbers(canonicalizeMetricsInput(raw))),
  // datas ISO, strings NFC, etc.
});
const payload = stableStringify(ded);
const hash = sha3_512(payload);

```

C. Verificador mínimo (LLM-V)

```

const verifierPrompt = `Responda APENAS JSON.
Pergunta: O texto abaixo contém algum PII da lista [emails, telefones, CPF]?
Texto: ""${escapedText}""
Saída: {"violates": true|false, "reasons": ["..."]} `;

const {violates} = await callSmallModel(verifierPrompt, {maxTokens:64, temp:0});
if (violates) return BLOCK;

```

D. Transação Ledger + Outbox + Debug

```

await prisma.$transaction(async tx => {
  const log = await tx.auditLog.create({ data: dedWithHashes });
  await tx.tsaOutbox.create({ data: { auditLogId: log.id, status: 'queued' } });
  if (LOGGING_LEVEL==='debug') {
    await tx.auditLogDebug.create({ data: encryptDebug(log.id, rawIn,
rawOut) });
  }
});

```

perfeito — começando pelos dois primeiros passos combinados: **(1) schema.prisma consolidado** e **(2) tipos/validadores compartilhados (Zod) para BE/FE**. Entrego como *patches aplicáveis agora*.

1) schema.prisma (fundação estável + enums + outbox + debug)

Compatível com PostgreSQL. Os campos de debug são Bytes (vamos cifrar no app com “envelope encryption”). Índices e unicidades incluídos.

```
// ----- generator & datasource -----
```

```
generator client {  
  provider = "prisma-client-js"  
}
```

```
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}
```

```
// ----- enums canônicos -----
```

```
enum Decision {  
  ALLOW  
  REVIEW  
  BLOCK  
}
```

```
enum Mode {  
  FAST  
  DEEP  
}
```

```
enum TsaStatus {  
  queued  
  processing
```

```

    confirmed
    failed
}

// ----- modelos -----
model User {
    id          String      @id @default(uuid())
    email       String      @unique
    name        String?
    role        String      @default("AUDITOR") // ADMIN_COMPLIANCE | AUDITOR |
OPERATOR_HITL

    Policies    PolicyConfig[]
    AuditLogs    AuditLog[]
}

// policy do Ω (JSON de regras) com hash canônico
model PolicyConfig {
    id          String      @id @default(uuid())
    domain      String      @unique
    version     Float       @default(1.0)
    rules       Json
    policyHash  String      @db.VarChar(128)

    createdAt   DateTime    @default(now())
    updatedAt   DateTime    @updatedAt

    ownerId     String?
    Owner       User?       @relation(fields: [ownerId], references: [id])
}

// log canônico (DED) – “ouro de auditoria”
model AuditLog {
    id          String      @id @default(uuid())
    createdAt   DateTime    @default(now())

```

```

policyDomain          String
policyVersion          Float
policyHash             String

decision              Decision
ruleTriggered         String?
rationale              String?

// métricas canônicas (flat) vindas do MIR
metrics_input          Json

mode                  Mode
latency_ms            Int
llm_provider          String?

text_hash_input        String
text_hash_output_final String?

payload_hash_sha3_512  String      @unique

tsa_status             TsaStatus  @default(queued)
tsa_token              String?
tsa_authority_uri      String?

correlationId          String?

userId                 String?
User                   User?      @relation(fields: [userId], references:
[id])

@@index([policyDomain])
@@index([correlationId])
}

```

```
// outbox para TSA (idempotência + retry)
model TsaOutbox {
  auditLogId String @id
  status      TsaStatus @default(queued)
  attemptCount Int      @default(0)
  lastError   String?

  AuditLog AuditLog @relation(fields: [auditLogId], references: [id],
onDelete: Cascade)
}

// área de debug com dados sensíveis (cifrados pela app)
model AuditLogDebug {
  auditLogId String @id
  raw_text_input Bytes
  raw_text_output Bytes

  AuditLog AuditLog @relation(fields: [auditLogId], references: [id],
onDelete: Cascade)
}
```

Notas de migração (executar agora)

```
npx prisma generate
npx prisma migrate dev --name init_v12_foundation
```

2) Tipos compartilhados + Zod (monorepo @shared-types)

Crie um pacote interno `packages/shared-types` (ou `src/shared` se preferir monorepo simples). Isso blinda a forma dos objetos em BE/FE.

```
packages/
  shared-types/
    src/
      enums.ts
      metrics.ts
```

```
dossier.ts
policy.ts
api.ts
canon.ts
package.json
tsconfig.json
```

enums.ts

```
export const Decision = { ALLOW: 'ALLOW', REVIEW: 'REVIEW', BLOCK: 'BLOCK' } as const;
```

```
export type Decision = keyof typeof Decision;
```

```
export const Mode = { FAST: 'FAST', DEEP: 'DEEP' } as const;
```

```
export type Mode = keyof typeof Mode;
```

```
export const TsaStatus = { queued: 'queued', processing: 'processing',
confirmed: 'confirmed', failed: 'failed' } as const;
```

```
export type TsaStatus = keyof typeof TsaStatus;
```

metrics.ts (Zod do MetricsInput flat e estável)

```
import { z } from 'zod';
```

```
export const MetricsInputZ = z.object({
  pii_count: z.number().int().nonnegative(),
  compliance_keywords_found: z.array(z.string()),
  basic_sentiment_score: z.number(), // -1..1 (não reforçamos aqui)
  extracted_claims: z.array(z.string()),
  plugin_sentiment_score: z.number().nullable().default(null),
  plugin_sentiment_latency_ms: z.number().int().nullable().default(null),
  plugin_bias_score: z.number().nullable().default(null),
}).strict();
```

```
export type MetricsInput = z.infer<typeof MetricsInputZ>;
```

policy.ts (PolicyRule/Config do Editor + hash)

```
import { z } from 'zod';
```

```
export const PolicyRuleZ = z.object({
  name: z.string().min(1),
  priority: z.number().int().min(1),
  condition: z.string().min(1), // expressão (CEL/JSONLogic)
  action: z.enum(['ALLOW', 'REVIEW', 'BLOCK']),
  rationale: z.string().min(1),
}).strict();
```

```
export const PolicyConfigPostZ = z.object({
  rules: z.array(PolicyRuleZ).min(1),
}).strict();
```

```
export type PolicyRule = z.infer<typeof PolicyRuleZ>;
```

dossier.ts (DED/PoP-L canônico + validações)

```
import { z } from 'zod';
import { MetricsInputZ } from './metrics';
import { Decision, Mode, TsaStatus } from './enums';
```

```
export const DecisionDossierZ = z.object({
  policyDomain: z.string().min(1),
  policyVersion: z.number(),
  policyHash: z.string().min(1),

  decision: z.enum(['ALLOW', 'REVIEW', 'BLOCK']),
  ruleTriggered: z.string().nullable(),
  rationale: z.string().nullable(),

  metrics_input: MetricsInputZ,

  mode: z.enum(['FAST', 'DEEP']),
  latency_ms: z.number().int().nonnegative(),
  llm_provider: z.string().nullable(),
```

```

text_hash_input: z.string().min(32),
text_hash_output_final: z.string().min(32).nullable(),

correlationId: z.string().optional(),
}).strict();

export type DecisionDossier = z.infer<typeof DecisionDossierZ>;

```

api.ts (contratos das rotas principais)

```

import { z } from 'zod';
import { PolicyRuleZ } from './policy';
import { Decision, Mode, TsaStatus } from './enums';
import { DecisionDossierZ } from './dossier';

export const GetPolicyResponseZ = z.object({
  id: z.string(),
  domain: z.string(),
  version: z.number(),
  rules: z.array(PolicyRuleZ),
  policyHash: z.string(),
  createdAt: z.string(),
}).strict();

export const CreatePolicyRequestZ = z.object({
  rules: z.array(PolicyRuleZ).min(1),
}).strict();

export const AuditLogEntryZ = z.object({
  id: z.string(),
  createdAt: z.string(),
  decision: z.enum(['ALLOW', 'REVIEW', 'BLOCK']),
  rationale: z.string().nullable(),
  mode: z.enum(['FAST', 'DEEP']),
  tsa_status: z.enum(['queued', 'processing', 'confirmed', 'failed']),
}).strict();

```



```
export const AuditLogDetailZ = DecisionDossierZ.extend({
  id: z.string(),
  createdAt: z.string(),
  payload_hash_sha3_512: z.string(),
  tsa_status: z.enum(['queued', 'processing', 'confirmed', 'failed']),
  tsa_token: z.string().nullable().optional(),
  tsa_authority_uri: z.string().nullable().optional(),
  // campos de debug são opcionais e só aparecem com RBAC+level
  raw_text_input: z.string().optional(),
  raw_text_output: z.string().optional(),
}).strict();
```

canon.ts (utilitários de canonização determinística)

// canonicaliza números, ordena chaves, fixa precision – antes de hash

```
export function canonicalizeNumbers(obj: any): any {
  const f = (v: any): any =>
    typeof v === 'number' && Number.isFinite(v) ? Number(v.toFixed(6))
    : Array.isArray(v) ? v.map(f)
    : v && typeof v === 'object'
      ? Object.fromEntries(Object.keys(v).sort().map(k => [k, f(v[k])]))
      : v;
  return f(obj);
}
```

// garante presença de TODAS as chaves do MetricsInput (flat)

```
export function canonicalizeMetricsInput(partial: any) {
  return {
    pii_count: partial.pii_count ?? 0,
    compliance_keywords_found: partial.compliance_keywords_found ?? [],
    basic_sentiment_score: partial.basic_sentiment_score ?? 0,
    extracted_claims: partial.extracted_claims ?? [],
    plugin_sentiment_score: partial.plugin_sentiment_score ?? null,
    plugin_sentiment_latency_ms: partial.plugin_sentiment_latency_ms ?? null,
    plugin_bias_score: partial.plugin_bias_score ?? null,
  };
}
```

```
};  
}
```

Como usar no backend (Nest)

- Adicione o pacote no `tsconfig.paths` ou como dependência local.
- Em **LedgerService** (antes de gerar o hash):

```
import stableStringify from 'json-stable-stringify';  
import { DecisionDossierZ } from '@shared-types/dossier';  
import { canonicalizeMetricsInput, canonicalizeNumbers } from  
'@shared-types/canon';  
  
const ded = DecisionDossierZ.parse({  
  ...dadosFixos,  
  metrics_input: canonicalizeNumbers(  
    canonicalizeMetricsInput(metricsInput) // <- TODAS as chaves presentes  
  ),  
});  
  
const payload = stableStringify(ded);  
const hash = createSha3_512(payload);
```

Como usar no frontend (React)

- Importe `AuditLogEntryZ/AuditLogDetailZ` e valide as respostas da API:

```
const res = await apiFetch('/api/v1/logs');  
const logs = z.array(AuditLogEntryZ).parse(res.data);
```
